

## Rob The Block: Refactoring Report

### Code Smells :

- Level.Java -
  - God class:  
Level.java was acting as a god class for creating and storing information about all the levels. As a fix, we made it an abstract class and now each level extends it. Now each level is a class and stores its own information.
  - Wall data Switch Statement:  
A switch statement was being used to get details about the wall for each level. The switch statement was eliminated by making an abstract method for the job named getWallData().
  - Patrol Path switch statement:  
A switch was being used for patrol path. It was making the class extra long. Created a new factory class called patrol factory to eliminate the switch statement and improve the code.
  - Populate level Switch Statement:  
A switch statement was being used in order to populate each level's information. We eliminated the switch by making levels independent classes.
  - Long method:  
The load level method was more than a 100 lines initially, it has been refactored to be around 34 lines now.
- Burglar.java-
  - Long Method :  
The move method was doing way too much at once. It was calculating where to go, checking if the move was valid, updating the position AND collecting items all in one method. We broke it down by pulling out the position update and item collection parts into their own helper methods. This cut the method from around 27 lines down to 16 and made it way easier to understand what's happening.
  - Duplicated Code:  
We had the exact same item collection code copy-pasted in both the regular move method and the diagonal move method. Instead of maintaining the same logic in two places, we made both methods use a shared helper that handles position updates and item collection.
  - Primitive Obsession:  
The movement speed was just set to 240 with no explanation of what that number actually means. We changed it to say the burglar moves 7.5 tiles per second, and then calculate the pixel speed from that. Now when you look at the code, you can actually tell how fast the character is moving instead of trying to figure out what 240 pixels per second feels like.

- Game.java-
  - Long Method:

The render() method in Game.java was 350+ lines handling everything from input to rendering to UI. We extracted the game state screen rendering (START\_SCREEN, GAME\_OVER, WIN) into three separate methods: renderStartScreen(), renderGameOverScreen() and renderWinScreen(). This reduced render() by around 50 lines and made each game state self-contained.
  - Feature Envy:

The render method had all this homeowner movement timer code mixed in with everything else, which made it hard to follow. We pulled that logic out into its own method that handles when homeowners should move and whether they're chasing the player or just patrolling. This made render cleaner and put the homeowner behavior in one spot where it belongs.